Semantic Mirror

Model-Valued Reflection Beyond BusyLean's Contender 6

Research report prepared for Vladimir Reshetnikov

October 12, 2026 (America/Los_Angeles)

Abstract

We construct a specific computable natural number by reflecting over complete semantic libraries rather than merely over numerical outputs. A library exposes a reflection hierarchy, a generator of well-founded trees, a common seed, a numerical input, an independent tower amplifier, and a relative function family. The underlying raw finite-type W-language is exactly the one already used in BusyLean. A fixed rank-17 term in this unchanged syntax reconstructs and amplifies an older library's entire measured construction. A new reflection operator therefore exceeds that construction by an explicit large margin.

Structural recursion on generated W-trees is then lifted from numerical functions to whole libraries. We join all finite self-generated bootstrap rounds and bootstrap the resulting limit library once more. Writing $C(n)$ for the pinned repository's Contender-6 family and $N(n)$ for the resulting family, we prove $B^{\circ(n+2)}(C(n)) < N(n)$, where $B(x) = x + 1 + T(x+1, x+1)$ and $T(h+1, x) = 2^{T(h,x)}$. Thus the proposed entry $N(42)$ exceeds 44 successive self-sized tower amplifications of Contender 6. A stronger qualitative theorem gives eventual domination and non-representability in every finite predecessor library, and also in the limit library. The package supplies a standalone Lean implementation proposal, full mathematical proofs, an independent executable reference model, exact source provenance, and validation scripts.

Verification boundary. The mathematical arguments are given in this report. **The newly supplied Lean source has not been compiled: no Lean executable was available in the generating environment.** Python tests and upstream source-hash checks passed. Kernel acceptance, axiom/dependency audits, and the contest's compilation-time budgets remain unexecuted. This is a research proposal with proof source, not a certified contest admission.

$$B^{\circ 44}(\mathrm{Contender}_6) < \underbrace{N(42)}_{\texttt{BusyLean.contender_7}}$$

Pinned baseline: `86b360648163f0249eab79539438b6a83dfe8215`.

Definition: `lean/BusyLean/Contender7.lean`.

Model and proof: `lean/Mirror/Model.lean`; `lean/Mirror/Comparison.lean`.

Contents

1 The target and the nature of the advance 3

2 The inherited language and the exact baseline 4

3 Semantic models and their libraries 5

4 A small compiler for the entire preceding construction 6

5 Joining libraries without an oracle-monotonicity assumption 8

6 From joins to strict model comparisons 9

7 Self-generated, model-valued bootstrapping 10

8 Computability and well-foundedness 11

9 The explicit tower separation 12

10 The stronger separation between semantic libraries 13

11 What is new here, and what is deliberately not claimed 14

12 Implementation and reproducibility 15

13 Further research questions 17

14 Conclusion 18

A Theorem-to-source map 18

B Provenance and source boundaries 18

C A general amplification schema 19

1 The target and the nature of the advance

The baseline is the current adopted Contender 6 at the pinned BusyLean commit [1, 4], not merely the earlier Coq Contender 5. Its central ingredients are a total interpreter for finite types with W-types, reflection relative to a countable family of functions, and an index joining all trees produced by the language. The repository already provides exact comparisons with Contender 5 and proposal P5 [5]. Simply rebuilding a System-T maximum, or adding an arithmetic operation to the earlier contender, would not address the actual target.

The present construction changes the level at which reflection operates. An older stage is made available as a *library of higher-order operations*. Programs may supply it with arbitrary seeds and trees, generate new trees using it, and combine it with the other operations in the original language. The next stage takes a finite maximum over this enriched semantics. In particular, it exceeds a short program that reconstructs the older stage's completed measurement, but that one comparison program is only a small part of what is being reflected.

There are two independent distinctions to keep clear. First, the numerical inequality proved here is much stronger than $C_6 + 1 < N$, but it is only a conservative lower bound on a semantic construction. Second, a mathematical proof in an article and a successfully checked Lean proof are different artifacts. We provide the first and source intended to implement it; we do not claim the second has been obtained in this environment.

1.1 Precise comparison coverage

The direct mathematical comparison is against the pinned adopted C6, uniformly in its input. C5 and P5 then follow by transitivity using the repository's existing comparisons; a separate integration file records these two steps. The repository also retains proposals P1–P4, whose ranking in the upstream review uses informal ordinal estimates [6]. This report does *not* silently turn those estimates into exact finite-number theorems. Separate rigorous comparisons with those archived proposals remain an integration task. No claim is made to exceed every arbitrary auxiliary constant or every future addition to the repository.

The repository's stated rules require a constructive closed natural-number term, restrict axioms to `propext` and `Quot.sound`, prohibit the usual noncomputable or unchecked escape hatches, and impose 15-second and 60-second clean definition/proof budgets under Lean 4.34.0 [1]. The construction is designed for these rules. Its actual compilation and budget compliance remain to be tested.

1.2 Main theorem, before the definitions

Define

$$T(0, x) = x, \qquad T(h + 1, x) = 2^{T(h,x)}, \tag{1}$$

$$B(x) = x + 1 + T(x + 1, x + 1), \qquad B^{\circ 0}(x) = x. \tag{2}$$

Iteration is ordinary function composition: $B^{\circ(k+1)}(x) = B(B^{\circ k}(x))$. The final family N is constructed in Sections 3–7.

Theorem 1.1 (Uniform tower margin)**.** *For every $n \in \mathbb{N}$,*

$$B^{\circ(n+2)}(C(n)) < N(n),$$

where $C(n)$ is exactly `BusyLean.finalFamily n` at the pinned commit. Consequently $B^{\circ 44}(C(42)) < N(42)$ and $C(42) < N(42)$.

The proof is given in Section 9. The source counterpart is `Mirror.iterated_boost_lt_final`; the closed-instance counterparts are `BusyLean.contender_6_lt_contender_7` and `BusyLean.contender_7_tower_margin`.

2 The inherited language and the exact baseline

2.1 Finite types with well-founded branching

The inherited type syntax is

$$A ::= \mathsf{nat} \mid A \to A \mid \mathsf{wt}\, A.$$

Its interpretation satisfies

$$[\![\mathsf{nat}]\!] = \mathbb{N}, \qquad [\![A \to D]\!] = [\![A]\!] \to [\![D]\!], \qquad [\![\mathsf{wt}\, A]\!] = W([\![A]\!]),$$

where $W(X)$ has constructors

$$\mathsf{leaf} : W(X), \qquad \mathsf{node} : (X \to W(X)) \to W(X).$$

Only the countably branching type $\mathcal{T} = W(\mathbb{N})$ is needed for the outer schedules, but the interpreted language retains W-types over every one of its finite types. Its raw terms have nine constructors: variables, typed lambdas, application, zero, successor, natural recursion, W-leaf, W-node, and W-recursion. All definitions are taken unchanged from the repository's P5 language implementation [2].

Values are dependent pairs (A, v) with $v \in [\![A]\!]$. The interpreter is total on *raw*, possibly ill-typed terms: unsuccessful applications or casts follow the repository's default-value conventions. We do not replace these conventions with a different dynamic semantics. This matters because the baseline is a maximum over raw terms, not merely a maximum over well-typed terms.

The function $\mathsf{allTerms}(d)$ is a finite list built structurally in d. Type annotations, variable levels, and all term constructors are bounded by the same enumeration-depth parameter. We use this exact enumeration throughout. There is no filter for semantic totality and no halting test.

2.2 The old reflection algebra

Put $\mathcal{F} = \mathbb{N} \to \mathbb{N} \to \mathbb{N}$. Let $R_0 : \mathcal{F} \to \mathbb{N} \to \mathbb{N}$ denote the inherited `BusyLean.W.reflect`. If $f \in \mathcal{F}$, its definition is

$$R_0(f, n) = n + 1 + \max\left\{ \max_{i \le n} f(i, n),\ \max_{t \in \mathsf{allTerms}(n)} \mathsf{evalWith}(f, t) \right\}. \tag{3}$$

An empty term-list maximum is zero. All maxima in numerical computations are finite. The function-family variable in this original relative environment is at oldest-first level zero [3].

Let $S \in \mathcal{F}$ be the inherited `BusyLean.p5Family`:

$$S(i, n) = \mathsf{iterate}(\mathsf{diagonal}, n, n).$$

Its definition ignores i. It contains the existing P5 growth mechanisms and its explicitly retained old-language maximum guard. We reuse this common *family*; we do not use the named C6 natural number as an initial value.

For a general algebra $r : \mathcal{F} \to \mathbb{N} \to \mathbb{N}$, define the tree hierarchy

$$\mathsf{walk}(r, S, \mathsf{leaf}) = r(S), \tag{4}$$

$$\mathsf{walk}(r, S, \mathsf{node} f) = r\big(i \mapsto \mathsf{walk}(r, S, f(i))\big). \tag{5}$$

The second argument of every family is the numerical input. These are structural equations on $\mathcal{T}$.

2.3 Recovering the old number without using its name

Write $\mathsf{nthD}(l, i, d)$ for list lookup with a default. Let

$$g_0(k, i) = \mathsf{defTree}\big(\mathsf{nthD}(\mathsf{allTerms}(k), i, \mathsf{zero})\big),$$

where $\mathsf{defTree}$ is the original cast to $W(\mathbb{N})$ after closed interpretation. Put

$$I_0 = \mathsf{node}\big(k \mapsto \mathsf{node}(i \mapsto g_0(k, i))\big).$$

The old family is exactly

$$C(n) = \mathsf{walk}(R_0, S, I_0, n). \tag{6}$$

This is not a size estimate: it follows by structural induction equating `walk` with the original `stage`, together with the original definitions of `treesOfDepth` and `index` [4]. The Lean bridge is `number_initial`.

3 Semantic models and their libraries

Definition 3.1 (Model)**.** A model is a pair $m = (r_m, g_m)$ with

$$r_m : \mathcal{F} \to \mathbb{N} \to \mathbb{N}, \qquad g_m : \mathbb{N} \to \mathbb{N} \to \mathcal{T}.$$

Its hierarchy, raw index, and numerical measurement are

$$H_m(s, t, n) = \mathsf{walk}(r_m, s, t, n), \tag{7}$$

$$I_m = \mathsf{node}\big(k \mapsto \mathsf{node}(i \mapsto g_m(k, i))\big), \tag{8}$$

$$\nu_m(n) = H_m(S, I_m, n). \tag{9}$$

The initial model is $m_0 = (R_0, g_0)$, so $\nu_{m_0}(n) = C(n)$.

The abstract theorems below hold extensionally for arbitrary models of these types. *Computability* is a separate closure property: for the final number we instantiate models only by the explicit computable constructors specified here. We do not assume that every mathematical function of the record's field types is computable.

A model supplies a fixed typed library to the unchanged interpreter. Its six oldest-first variable levels have the following meanings.

Level	Value	Type and purpose
0	H_m	$\mathcal{F} \to \mathcal{T} \to \mathbb{N} \to \mathbb{N}$; the entire hierarchy, with arbitrary seed and tree.
1	g_m	$\mathbb{N} \to \mathbb{N} \to \mathcal{T}$; the whole tree generator.
2	S	$\mathcal{F}$; the common seed family.
3	n	$\mathbb{N}$; the current external numerical input.
4	B	$\mathbb{N} \to \mathbb{N}$; the independent tower amplifier.
5	f	$\mathcal{F}$; the current relative function family.

The actual environment list is in the reverse order, because the interpreter stores the newest binding first but resolves variables by oldest-first levels. Appending a lambda-bound variable therefore creates level 6, then level 7, without changing any of the six library levels. Confusing this with ordinary nearest-binder de Bruijn indexing would invalidate the comparison program.

Let

$$E(m, f, n, t) = \mathsf{score}(m, f, n, t)$$

be the numerical projection of the original interpreter in this environment. A non-natural result contributes zero. We use $0_{\mathcal{F}}$ for the constantly zero family.

Remark 3.2 (What has actually been added)**.** No new term constructor, quotation operation, or evaluator primitive is added to the object language. The extension is in the *values assigned to its free variables*. These are concrete total higher-order functions. The measurement ν_m is not itself a primitive library symbol: a short ordinary program must assemble it from H_m, g_m, S, and n.

4 A small compiler for the entire preceding construction

4.1 The witness program

In informal typed notation define

$$p_I = \mathsf{node}(\lambda k : \mathbb{N}.\, \mathsf{node}(\lambda i : \mathbb{N}.\, g_m(k, i))), \tag{10}$$

$$p_V = H_m(S, p_I, n), \tag{11}$$

$$p_B = B(p_V). \tag{12}$$

These expressions are templates in the six library variables, not special primitives inserted into the enumeration. Their exact raw Lean syntax is:

```
def indexProgram : Term :=
  .app (.wnode .nat)
    (.lam .nat treeTy
      (.app (.wnode .nat)
        (.lam .nat treeTy
          (.app (.app (.var 1) (.var 6)) (.var 7)))))

def numberProgram : Term :=
  .app (.app (.app (.var 0) (.var 2)) indexProgram) (.var 3)

def boostProgram : Term := .app (.var 4) numberProgram
```

There is one such finite syntax tree for all models, all relative families, and all inputs. In particular, there is no numeral encoding $C(42)$ and no special term referring to the named previous contender.

4.2 Same-type W-casts are not judgmental identities

The upstream cast from $W(A)$ to the same W-type applies a branch reindexing map. Even if the map on A is an identity, the resulting tree is not in general *definitionally* the original arbitrary tree. This is a genuine proof engineering issue, not a numerical discrepancy.

Lemma 4.1 (Identity reindexing)**.** *For every type X and every $t \in W(X)$, $\mathsf{wmap}(\mathrm{id}, t) = t$ extensionally.*

Proof. Use W-induction. The leaf case is the defining equation. For $t = \mathsf{node} f$, the left side is $\mathsf{node}(x \mapsto \mathsf{wmap}(\mathrm{id}, f(x)))$. The induction hypotheses identify every branch with $f(x)$. Function extensionality identifies the branch functions, and congruence identifies the nodes. □

Lemma 4.2 (Same-type cast identity)**.** *For every finite type A and $v \in [\![A]\!]$, the original* $\mathsf{castValue}(A, v)$ *is extensionally equal to v.*

Proof. Prove both directions of the original $\mathsf{castPair}(A, A)$ together by induction on A. For nat both functions are identities. The arrow case composes the already-identity domain and codomain casts, and the Boolean same-type test is true by structural induction. The W-case invokes Lemma 4.1 after the branch-type induction hypothesis. Projecting the first component proves the statement. □

This use of function extensionality is compatible with the stated allowance of `Quot.sound`; it is not an assertion that the resulting axiom audit has already been executed. The numerical definitions do not depend on these cast-equality proof terms.

Lemma 4.3 (Parametric compiler correctness)**.** *For arbitrary $h : \mathcal{F} \to \mathcal{T} \to \mathbb{N} \to \mathbb{N}$, $g : \mathbb{N} \to \mathbb{N} \to \mathcal{T}$, $s \in \mathcal{F}$, $a : \mathbb{N} \to \mathbb{N}$, $f \in \mathcal{F}$, and $n \in \mathbb{N}$, interpret p_B with library values (h, g, s, n, a, f). Its numerical result is*

$$a\left(h\left(s, \mathsf{node}(k \mapsto \mathsf{node}(i \mapsto g(k, i))), n\right)\right).$$

Consequently $E(m, f, n, p_B) = B(\nu_m(n))$.

Proof. At the outer lambda in p_I, the new bound argument is at oldest-first level 6. At the inner lambda, its argument is at level 7; level 6 still denotes the outer argument, and level 1 still denotes g. Thus the innermost pair of applications has value $g(k, i)$. Lemma 4.2 removes the homogeneous casts introduced by the applications and typed lambdas. Applying the two node constructors yields exactly the displayed two-level tree. Levels 0, 2, and 3 then give h, s, and n in p_V. Level 4 applies a to the numerical result. Level 5 is unused, so the equation is independent of f. □

4.3 A rank certificate, rather than a gigantic enumeration

Define the type rank by

$$\tau(\mathsf{nat}) = 1, \quad \tau(A \to D) = 1 + \max(\tau(A), \tau(D)), \quad \tau(\mathsf{wt}\, A) = 1 + \tau(A).$$

Define the term rank by the following equations:

$$\begin{aligned} \rho(\mathsf{var}\, i) &= i + 2, \\ \rho(\mathsf{lam}\, A\, D\, t) &= 1 + \max(\tau(A), \tau(D), \rho(t)), \\ \rho(\mathsf{app}\, t\, u) &= 1 + \max(\rho(t), \rho(u)), \\ \rho(\mathsf{zero}) = \rho(\mathsf{succ}) &= 1, \\ \rho(\mathsf{natrec}\, A) = \rho(\mathsf{wzero}\, A) = \rho(\mathsf{wnode}\, A) &= 1 + \tau(A), \\ \rho(\mathsf{wrec}\, A\, D) &= 1 + \max(\tau(A), \tau(D)). \end{aligned}$$

The variable convention $i + 2$, rather than $i + 1$, matches the original $\mathsf{natsUpTo}$ call at the preceding enumeration depth.

Lemma 4.4 (Constructive enumeration completeness)**.** *If $\tau(A) \le d$, then $A \in \mathsf{allTypes}(d)$. If $\rho(t) \le d$, then $t \in \mathsf{allTerms}(d)$.*

Proof. For types, use constructor induction with arbitrary d. Positive rank rules out $d = 0$. At depth $d + 1$, the required constituent types have rank at most d; the arrow cross-product or W-type map therefore contains the desired type. The natural type is inserted directly.

For terms, use the same induction. If $t = \mathsf{var}\, i$ and the outer depth is $d+1$, $i+2 \le d+1$ gives $i < d$, which places i in $\mathsf{natsUpTo}(d)$. A lambda is in the product of the two annotation-type lists and the body-term list. An application is in the product of the two term lists. Zero and successor are inserted directly. The natural and W recursors and the W constructors occur in the corresponding type-list maps or type-pair cross-product. These are exactly the branches in the original enumerator, so every constructor case is covered. □

The concrete rank calculation is small:

Object	Rank
$\mathsf{wt}\,\mathsf{nat}$	2
$g(k,i)$ with variable levels $1, 6, 7$	10
Inner lambda $\lambda i.\, g(k,i)$	11
Inner node	12
Outer lambda	13
p_I	14
p_V	16
p_B	17

Consequently $p_B \in \mathsf{allTerms}(n+17)$ for every n. This certificate requires traversing only the fixed program's syntax. It never constructs $\mathsf{allTerms}(17)$. We do not claim that 17 is the smallest possible comparison-compiler overhead among all alternative encodings.

5 Joining libraries without an oracle-monotonicity assumption

5.1 Two evaluation channels

Let $b(n) = n + 17$ and define

$$P(m,f,n) = \max_{t \in \mathsf{allTerms}(b(n))} E(m,f,n,t), \tag{13}$$

$$L(m,f,n) = \max\{P(m,0_{\mathcal{F}},n), P(m,f,n)\}. \tag{14}$$

The zero-family channel preserves the semantics of an older closed library; the relative channel allows programs to inspect the newly supplied family.

This separation avoids an invalid argument: pointwise increasing a function family need not increase every program's output. A program that returns one when $f(0,0) = 0$ and zero otherwise already decreases when the constant-zero family is replaced by the constant-one family. Such a zero test is definable using the inherited natural recursor. This counterexample is about a *program*; it does not by itself prove any particular finite reflection *maximum* nonmonotone. Our argument needs neither conclusion about the whole maximum: it retains the relevant semantics exactly.

5.2 The numerical reflection component

For a family of models $\mathbf{m} = (m_i)_{i \in \mathbb{N}}$ define

$$R_{\mathbf{m}}(f,n) = n + 1 + \max\left\{R_0(f,n),\ \max_{i \le n} L(m_i,f,n)\right\}. \tag{15}$$

Only the finite prefix $i \le n$ is evaluated. Retaining R_0 also preserves the original relative interpreter and its visible-oracle bounds. For the principal comparison theorem it is enough that the second maximum is present; the first component strengthens rather than weakens the definition.

Lemma 5.1 (Program bounds at a join)**.** *If $i \le n$ and $\rho(t) \le n + 17$, then for every $f \in \mathcal{F}$,*

$$E(m_i, 0_{\mathcal{F}}, n, t) < R_{\mathbf{m}}(f, n), \qquad E(m_i, f, n, t) < R_{\mathbf{m}}(f, n).$$

Proof. Lemma 4.4 puts t in the finite term list. Its score is bounded by the corresponding P, hence by L, hence by the prefix maximum over $i \le n$, hence by the final maximum in (15). Adding $n + 1$ strictly exceeds each such bound. The proof never changes the oracle of the program whose score is being bounded. □

5.3 The tree-generation component

For each model define the tree-valued interpretation

$$Q(m, k, t) = \mathsf{castValue}\,(\mathsf{wt}\,\mathsf{nat}, \mathsf{interpret}(\mathsf{libraryEnv}(m, 0_{\mathcal{F}}, k), t).\mathsf{value})\,.$$

Define a generator for the joined model by

$$G_{\mathbf{m}}(k, i) = \mathsf{node}\,(j \mapsto Q\,(m_j, k, \mathsf{nthD}(\mathsf{allTerms}(k), i, \mathsf{zero})))\,. \tag{16}$$

Finally set

$$\mathsf{J}(\mathbf{m}) = (R_{\mathbf{m}}, G_{\mathbf{m}}).$$

This countable join is an ordinary W-node with a branch function. It is not a numerical maximum over infinitely many model values.

The tree channel and numerical channel serve different purposes. The numerical channel guarantees explicit inequalities at an input. The tree channel makes the enriched libraries available for creating the next self-generated tree schedule. There is no fixed global ordinal-notation system that must first be shown to contain those schedules.

Proposition 5.2 (Exact tree-program retention)**.** *If $\mathsf{nthD}(\mathsf{allTerms}(k), i, \mathsf{zero}) = t$, then branch j of $G_{\mathbf{m}}(k, i)$ is exactly $Q(m_j, k, t)$.*

Proof. Substitute the lookup equality into (16). No monotonicity, rank comparison, or approximation of a tree is involved. □

Every term has a finite enumeration rank and therefore some finite list position. Thus every tree program in every selected library is retained at a finite generator address, although the enclosing node keeps the selected library as a separate branch. The Lean file records the exact lookup version as `tree_slot`.

6 From joins to strict model comparisons

Theorem 6.1 (A join exceeds an amplified whole child construction)**.** *For $i \le n$,*

$$B(\nu_{m_i}(n)) < \nu_{\mathsf{J}(\mathbf{m})}(n).$$

Proof. Apply Lemma 5.1 to the fixed term p_B, whose rank is 17. By Lemma 4.3, its score in child library m_i is $B(\nu_{m_i}(n))$. This is below $R_{\mathbf{m}}(f, n)$ for *every* relative family f.

The raw index of the joined model is a node. Therefore the root equation for walk expresses $\nu_{\mathsf{J}(\mathbf{m})}(n)$ as $R_{\mathbf{m}}(f_*, n)$ for its particular family of child hierarchy functions f_*. Instantiate the already-proved bound at f_*. There is no need to compare f_* with any older family. □

Theorem 6.2 (Eventual program domination)**.** *For every fixed child index i and fixed raw term t,*

$$n \geq \max\{i, \rho(t)\} \quad \Longrightarrow \quad E(m_i, 0_{\mathcal{F}}, n, t) < \nu_{\mathsf{J}(\mathbf{m})}(n).$$

Proof. The input bound implies both $i \leq n$ and $\rho(t) \leq n + 17$. Use the first inequality of Lemma 5.1 with the relative family occurring at the root of the joined model's measurement, just as in the previous proof. □

Corollary 6.3 (Non-representability in a child library)**.** *There is no fixed term t such that $E(m_i, 0_{\mathcal{F}}, n, t) = \nu_{\mathsf{J}(\mathbf{m})}(n)$ for all n.*

Proof. Choose $n = \max\{i, \rho(t)\}$. The proposed equality contradicts Theorem 6.2 at this very input. □

The important content is the quantification over arbitrary fixed terms in the whole older library, not only over the three comparison templates. Such a term may apply H_{m_i} to entirely different seeds and trees, exploit W-recursors at higher finite types, and call the independent amplifier many times. The next library still eventually exceeds its output. The threshold is a simple syntactic bound and does not depend on how long the program runs.

7 Self-generated, model-valued bootstrapping

7.1 A tree fold returning whole models

For $m \in \mathcal{M}$ define

$$\mathsf{Fold}(m, \mathsf{leaf}) = m, \tag{17}$$

$$\mathsf{Fold}(m, \mathsf{node} f) = \mathsf{J}(i \mapsto \mathsf{Fold}(m, f(i))). \tag{18}$$

This is a structural recursion on $\mathcal{T}$ with codomain $\mathcal{M}$. The crucial change from the old forest reflection is that each recursive result contains *both* a new reflection operator and a new tree generator. Later programs can invoke the older complete hierarchy, not just observe an older number.

The bootstrap schedule for a model is

$$\Gamma_m = \mathsf{node}(c_m), \tag{19}$$

$$c_m(0) = \mathsf{leaf}, \tag{20}$$

$$c_m(k+1) = \mathsf{node}(i \mapsto g_m(k, i)). \tag{21}$$

Define $\mathsf{Boot}(m) = \mathsf{Fold}(m, \Gamma_m)$. Branch zero retains the exact entire model m. All other branches come from its own tree generator. The guard avoids having to locate a particular tree program at a potentially gigantic enumeration-list position merely to establish the predecessor comparison. It does not replace the generated-tree component.

Theorem 7.1 (Bootstrap comparison)**.** *For every m and n,*

$$B(\nu_m(n)) < \nu_{\mathsf{Boot}(m)}(n).$$

Moreover, every fixed program of the closed m-library is eventually below $\nu_{\mathsf{Boot}(m)}$.

Proof. By (18), $\mathsf{Boot}(m)$ is the join of the family $i \mapsto \mathsf{Fold}(m, c_m(i))$. At index zero this family is exactly m. Theorem 6.1 applies because $0 \le n$ for every input. The same instantiation of Theorem 6.2 gives the program statement. □

7.2 All finite rounds, followed by a limit bootstrap

Set

$$m^{[0]} = m, \qquad m^{[k+1]} = \mathsf{Boot}(m^{[k]}), \tag{22}$$

$$m_\infty = \mathsf{J}(k \mapsto m_0^{[k]}), \qquad m_* = \mathsf{Boot}(m_\infty). \tag{23}$$

The proposed family and closed number are

$$N(n) = \nu_{m_*}(n), \qquad N = N(42). \tag{24}$$

The input 42 is applied only at the end. Every construction leading to it is uniform and parameter-free apart from the pinned, explicitly given initial library and the small amplifier definition.

```
def rounds (m : Model) : Nat -> Model
  | 0 => m
  | k + 1 => bootstrap (rounds m k)

def limitModel : Model := join (rounds initialModel)
def finalModel : Model := bootstrap limitModel
def finalNumber (n : Nat) : Nat := number finalModel n

-- In namespace BusyLean:
def contender_7 : Nat := Mirror.finalNumber 42
```

The arrows in this displayed excerpt are ASCII equivalents of the source's Unicode arrows. The full code is in the archive; no placeholder in this excerpt denotes an unimplemented operation there.

8 Computability and well-foundedness

The word "computable" should do real work in a large-number proposal. It is not enough to observe vacuously that any particular natural number could be printed by some program. Here there is an explicit uniform, terminating construction of the family $N(n)$ from the given finite definitions.

Proposition 8.1 (Constructive closure of the model operations)**.** *The initial model is computably presented. A uniformly computably presented family of models has a computably presented join. Given a computably presented model and tree, its model fold is computably presented. Consequently every finite bootstrap round, the limit model, the final model, and $N(n)$ are computable in principle.*

Proof. The inherited raw interpreter recurses structurally on finite syntax. Its natural and W recursors are total recursors; its cast operation recurses on finite type syntax and, where necessary, on W-trees.

The shared seed uses only these operations and structural recursion on natural iteration counts. Thus the initial reflection algebra and tree generator are computable from finite input descriptions.

For a join, the numerical reflection component enumerates finitely many indices $i \leq n$ and finitely many raw terms of depth $n + 17$. Every term is interpreted in a computably supplied library. There is no search for a proof of termination, and no wait for an unspecified program to halt. The tree component creates a W-node with a computable branch function; to request branch j is to interpret one specified term in one specified child library. Creating such a branch function does not require listing all its values.

The fold in (18) is precisely the recursor of $W(\mathbb{N})$ with result type $\mathcal{M}$. Its recursive arguments are the results at the immediate subtrees $f(i)$. The guarded index is built from the already-given generator of m, before this fold is invoked. It does not call the bootstrap being defined. Thus bootstrapping is composition of total functions followed by a structural recursion, not a circular fixed-point definition.

Finite rounds are a natural-number recursion. The family $k \mapsto m_0^{[k]}$ is uniformly computable, so the join closure applies to m_∞. Apply the bootstrap closure once more and then the total hierarchy measurement. This proves the claim. □

A countably branching W-tree may have an enormously complicated shape, but its inductive presentation is not an arbitrary possibly ill-founded graph. The recursion principle is the termination argument. Lean's documented structural-recursion mechanism translates accepted definitions to recursor applications [7]. Whether the particular source elaborates as intended still requires running the specified toolchain; the mathematical recursor construction does not substitute for that test.

8.1 What is not being used

The numerical construction does not use a Busy Beaver oracle, a semantic predicate selecting all total Lean programs, a nonconstructive maximum, or a universal interpreter for the language containing its own full semantics. Every finite maximum ranges over the old fixed raw syntax. The increase in strength comes from a hierarchy of explicitly built semantic libraries. Nor is there a partial-function shortcut that supplies a default numerical answer to an unproved termination problem.

All supplied numerical source is ordinary `def` code. The source scan finds no prohibited escape-hatch tokens outside comments and strings. A stronger transitive axiom/dependency audit is provided separately because a lexical scan alone cannot establish those properties.

9 The explicit tower separation

Lemma 9.1 (Elementary amplifier properties)**.** *For all natural x, $x < B(x)$. Also, $a \leq b$ implies $B(a) \leq B(b)$. Consequently $x \leq B^{\circ k}(x)$ for every k.*

Proof. The first assertion follows from the summand $x + 1$. Exponentiation by two is monotone and satisfies $y < 2^y$ for all natural y, including zero. Induction on the tower height therefore shows monotonicity in its top argument. The inequality $T(h, x) \leq T(h + 1, x)$ follows from $y < 2^y$ with $y = T(h, x)$; iterating it gives monotonicity in the height.

If $a \leq b$, then

$$T(a + 1, a + 1) \leq T(a + 1, b + 1) \leq T(b + 1, b + 1).$$

Add the monotone term $a + 1 \leq b + 1$. Finally induct on k, using $x < B(x)$ at the inductive step. □

Lemma 9.2 (Finite-round lower bound)**.** *For every model m and natural k, n,*

$$B^{\circ k}(\nu_m(n)) \le \nu_{m^{[k]}}(n).$$

Proof. For $k = 0$ this is equality. Suppose it holds for k. By Lemma 9.1 and Theorem 7.1,

$$B^{\circ(k+1)}(\nu_m(n)) \le B(\nu_{m^{[k]}}(n)) < \nu_{m^{[k+1]}}(n).$$

Discarding strictness proves the induction step. □

Proof of Theorem 1.1. Fix n and abbreviate $A_k = \nu_{m_0^{[k]}}(n)$ and $L_n = \nu_{m_\infty}(n)$. By Lemma 9.2, $B^{\circ n}(C(n)) \le A_n$. At the limit join, child index n is visible at input n. Theorem 6.1 gives $B(A_n) < L_n$. Hence

$$B^{\circ(n+1)}(C(n)) \le B(A_n) < L_n.$$

The final bootstrap satisfies $B(L_n) < N(n)$. Apply monotonicity of B to the preceding bound to obtain

$$B^{\circ(n+2)}(C(n)) \le B(L_n) < N(n).$$

This proves the uniform inequality. Set $n = 42$ for the advertised 44-fold margin, and use $C(n) \le B^{\circ(n+2)}(C(n))$ for strict domination of the unamplified baseline. □

9.1 Why "44" is not an arbitrary advertised height

The number 44 is a count of *repeated self-sized tower transformations.* At each application of B, both the tower height and its top argument are chosen from the already enormous previous value. It is not a tower of fixed height 44. The count comes from 42 bootstrap rounds visible in the limit comparison, one reflection at the limit join, and one more final bootstrap.

For orientation only, the first three tiny amplifier values are

$$B(0) = 3, \qquad B(1) = 18, \qquad B(2) = 2^{256} + 3.$$

No such display approximates the proposed entry. The full model construction also ranges over many programs and generated trees not used in this lower bound.

9.2 The proof avoids numerical normalization

Every substantial theorem is proved for abstract m, n before being specialized. The final comparison with 42 is merely an application of the uniform theorem. A checker should never need to reduce $C(42)$ or $N(42)$. The same principle governs compiler correctness: it is first proved for abstract h, g, s, n, a, f. Replacing those variables by concrete giant semantic objects inside a simplification proof would invite catastrophic evaluation.

10 The stronger separation between semantic libraries

Theorem 10.1 (Beyond every finite bootstrap library)**.** *For every $k \in \mathbb{N}$ and raw term t, if $n \ge \max\{k, \rho(t)\}$, then*

$$E(m_0^{[k]}, 0_{\mathcal{F}}, n, t) < N(n).$$

Thus no fixed program in any finite bootstrap library represents N at all inputs.

Proof. Theorem 6.2 for the limit join gives $E(m_0^{[k]}, 0_{\mathcal{F}}, n, t) < \nu_{m_\infty}(n)$ at the indicated inputs. The final bootstrap strictly exceeds its input model's measurement by Theorem 7.1 and $x < B(x)$. Compose the inequalities. For non-representability, instantiate at $n = \max\{k, \rho(t)\}$. □

Theorem 10.2 (Beyond the limit library itself)**.** *Every fixed program in the closed m_∞-library is eventually below N. In particular, no such program represents N at all inputs.*

Proof. Apply the program part of Theorem 7.1 directly to m_∞. It applies whenever $\rho(t) \leq n + 17$, and hence whenever $n \geq \rho(t)$. An equality at all inputs contradicts the bound at $n = \rho(t)$. □

This also yields a useful general obstruction to interpreting the strengthened family inside an older fixed library: any proposed representation must fail on a completely explicit sufficiently large input. The result is relative to the stated six-symbol semantic interface. It does not assert that the family is uncomputable, or that it cannot be written in Lean; its definition here is precisely a larger, externally constructed Lean expression.

No assumption that $C(n)$, $N(n)$, or arbitrary $E(m, f, n, t)$ is monotone in n occurs in these proofs. The only needed monotonic function is the independent amplifier B. This is a significant simplification compared with a strategy that tries to compare arbitrary reflection trees by pointwise oracle order.

11 What is new here, and what is deliberately not claimed

The construction combines three ideas in a concrete package: a constant-cost compiler for a full preceding hierarchy measurement, a structural tree fold whose outputs are whole semantic models, and a further traversal through the trees generated by the limit library itself. The resulting comparison and non-representability arguments are exact and avoid ordinal calibration. This is the proposed contribution of this report; a broad literature-priority claim has not been established.

The root compiler theorem already proves a strong lower bound even if one ignores much of the elaborate generated-tree schedule. That is a virtue for verification, but it also sets a boundary on what has been proved. We have not identified an exact ordinal for the whole construction, proved a sharp fast-growing-hierarchy equivalence, or quantified the additional advantage contributed by every nonzero generated-tree branch. The non-representability theorems concern the actual enriched libraries and are stronger evidence of structural separation than an unsupported ordinal label would be.

The new entry is not a claimed largest computable number. Once its description has been fixed, larger computable descriptions can be made. The relevant achievement is a substantial, reproducibly specified advancement over the actual current benchmark, together with the proof obligations that make the comparison meaningful.

11.1 The no-laziness issue

A mere name audit can be evaded by inlining a previous winner, so it is not a complete test of mathematical quality. Here the explicit advance is supported by the all-program semantic separation and the new model-valued recursion. Nevertheless, dependency hygiene is still useful and is made inspectable.

The intended dependency closure of `contender_7` excludes the named `BusyLean.contender_6`, `BusyLean.finalFamily`, and `BusyLean.index`, as well as P5's named contender and the three proof-side comparison-template constants. It retains the original raw interpreter, `BusyLean.W.reflect`,

`BusyLean.defTree`, `BusyLean.nthD`, and `BusyLean.p5Family`. The inherited old-language maximum guard inside that seed is not removed or concealed.

The definition module imports `BusyLean.Contender6` to obtain those shared helpers. Importing a module containing the old winner is different from having that winner in the transitive *term* dependency closure. The supplied Lean audit checks the latter. It has not been run, so the report records this as an intended invariant plus a direct-source check, not as a verified kernel fact.

12 Implementation and reproducibility

12.1 Module decomposition

Definitions and proofs are separated to avoid charging proof dependencies to the definition phase and to avoid accidentally evaluating comparison terms while defining the entry. The core numerical modules import only the inherited core-only sources. The proof-side modules may import the bundled Lean library for ordinary proof tactics; there is no Mathlib dependency.

Module	Role
`Mirror/Model.lean`	Models, libraries, joins, tree folds, bootstrap, final family.
`BusyLean/Contender7.lean`	Closed input-42 definition.
`Mirror/Compiler.lean`	Fixed syntax templates and rank functions.
`Mirror/Enumeration.lean`	Rank-based membership certificates.
`Mirror/Semantics.lean`	Cast identities, compiler semantics, exact baseline bridge.
`Mirror/Bounds.lean`	Finite-max bounds, model comparisons, non-representability.
`Mirror/Growth.lean`	Tower and amplifier inequalities.
`Mirror/Comparison.lean`	Uniform and closed quantitative comparisons.
`Mirror/Audit.lean`	Axiom whitelist and forbidden dependency closure.
`Mirror/Smoke.lean`	Small ranks and amplifier values only.

The archive vendors only three exact Lean dependency files: the original P5 `Definitions.lean`, `BusyLean/Reflection.lean`, and `BusyLean/Contender6.lean`. Their Git blob hashes agree byte-for-byte with the pinned repository. The standalone package is not a replacement repository; its integration instructions explicitly avoid overwriting the existing Lake configuration, library roots, or previous comparison files.

12.2 What was tested

The independent Python reference model reproduces the raw constructors, typed values, original cast conventions, oldest-first environments, and the original finite enumerators. Its actual run produced the following counts.

Depth	Types	Raw terms
0	0	0
1	1	2
2	3	13
3	13	308
4	183	147,129

At these depths the tests check the relevant rank bounds, membership behavior, and enumeration inclusion. They do *not* interpret all 147,129 depth-4 terms. A recursively checked certificate verifies the rank-17 compiler without constructing the depth-17 list.

The compiler semantics were also tested on 480 generic environments. These vary abstract hierarchy, generator, seed, input, amplifier, and relative-family values; selected tree paths are inspected to detect binder-order or tree-cast errors. Further checks cover 24 sampled identity-cast tree paths, a program whose output decreases under a larger family, and 100 bounded tower monotonicity cases. The tiny original family value $C(0) = 3$ is checked. No new model maximum, full contender, or full rank-17 enumeration is evaluated.

These are falsification-oriented engineering checks, not substitutes for universal proofs. In particular, finite tree-path sampling is not function extensionality, and interpreter examples do not certify every Lean elaboration. The full machine-readable results are retained in `evidence/`.

12.3 What remains unverified

No Lean executable was available, and toolchain download attempts did not succeed. Therefore the new source has not passed elaboration, code generation, kernel checking, axiom auditing, or dependency-closure auditing. The requested clean-build timing budgets are unmeasured. We do not infer compilation success from plausible source or from Python test success.

The most delicate proof-side points are the dependent sigma-value interpreter simplifications, homogeneous W-casts, and rank-membership proofs against the current core-library lemma interface. Their mathematical arguments are given above; the implementation may still require local elaboration repairs. There are no intentionally admitted proof obligations in the new source.

12.4 Reproduction commands and admission gates

From the archive root:

```
python3 python/test_reference.py
python3 scripts/source_audit.py
python3 scripts/check_lean.py
python3 scripts/build_pdf.py
```

The Lean checker requires version 4.34.0, accepts an explicit executable path, creates a clean build directory, compiles definition modules first, then proof modules, and finally runs the audit and safe smoke tests. It reports missing Lean as `NOT_RUN` with a nonzero exit code. It never treats unavailable verification as a pass. It charges audit time to the proof budget conservatively. The supplied script itself must be validated on an actual Lean installation; it is not a repair mechanism for failed proofs.

Before accepting the entry, require actual successful checks of the numerical definitions, all comparison and non-representability theorems, the allowed-axiom whitelist, the forbidden-name dependency closure, and the clean timing budgets. The optional full-repository file then chains C5 and P5 comparisons.

Do not evaluate the entry. Even input zero of the new family has a reflection enumeration budget of 17. The smoke tests intentionally reduce only tiny amplifier inputs and short syntax ranks. Checking a generic theorem is categorically different from computing the gigantic numbers it compares.

13 Further research questions

1. Complete the machine validation and minimize proof cost. The immediate task is to run Lean 4.34.0, repair any elaboration issues without changing the semantics, and collect actual dependency and timing evidence. Replace broad simplification with explicit parametric interpreter lemmas where needed. This is an essential completion criterion, not a claim of an already finished formalization.

2. Compile all archived proposals into an exact comparison library. Produce translations or exact semantic embeddings for P1–P4, with explicit syntax costs and finite-input bounds. This would replace the review's heuristic ranking by exact comparison theorems and close the remaining scope gap in this package.

3. Quantify the extra strength of generated-tree branches. The certified tower margin uses a deliberately easy visible branch. Prove lower bounds that exploit nonzero generated-tree branches, rather than merely the predecessor guard. A useful starting point is an address-preserving simulation relation on model-valued folds, designed to preserve exact library semantics rather than pointwise approximate arbitrary oracles.

4. Find the smallest effective compiler overhead. The current fixed program has rank 17. Alternative environment layouts, index encodings, or typed compilation may lower this cost. Prove optimality only relative to an explicit encoding and allowed library signature; a shorter-looking informal lambda term is not by itself an enumeration-depth certificate.

5. Separate semantic strength from numerical amplifier choice. Generalize the construction to an arbitrary supplied monotone expansive amplifier A. The same compiler and maximum argument should yield $A^{\circ(n+2)}(C(n)) < N_A(n)$ with unchanged syntax rank. This isolates the semantic-library advance from the chosen elementary tower certificate. The parametric compiler theorem already supplies most of the needed proof.

6. Develop a typed front end with a semantics-preserving raw translation. An intrinsically typed representation could simplify compiler proofs and avoid repeated cast reasoning. It must translate into the exact existing raw interpreter with explicit rank control, rather than replacing the baseline semantics silently. A logical relation would be a natural tool for this task.

7. Study reflection monotonicity precisely. Determine which restricted program fragments admit oracle monotonicity and whether any monotone envelope of the full finite maximum is useful. The individual zero-test counterexample does not settle monotonicity of the whole maximum. A theorem addressing that distinction could support sharper tree comparison methods without compromising the current exact-library argument.

8. Seek calibrated, conditional ordinal bounds. Only after fixing a precise notation system, fundamental sequences, and simulation theorem should one attach a fast-growing-hierarchy label to this construction. Such a result would be explanatory; it is not needed for the exact comparisons proved here. A credible calibration must account for both the interpreted language and the model-valued bootstrap layer.

14 Conclusion

The proposed number is specified by a finite, constructive chain of semantic operations. Its strongest established mathematical feature is not merely its 44-fold self-sized tower margin over the adopted Contender 6, but its eventual separation from every fixed program in each finite predecessor library and from every fixed program in the limit library. The entire mechanism uses the existing W-language with richer executable environments, and all numerical maxima remain finite.

The report separates this mathematical achievement from the uncompleted machine-verification step. The source, exact dependency snapshot, independent tests, and reproducible checks make that remaining step concrete and inspectable. The resulting package is a substantial proposed advancement over the current benchmark, with neither an unsupported optimality claim nor an invented kernel-verification claim.

A Theorem-to-source map

Mathematical item	Lean declaration in `BusyLean.Mirror` unless qualified
Identity tree reindexing	`wmap_id`
Same-type cast identity	`castPair_self`, `castValue_self`
Compiler semantics	`eval_boostProgram`, `score_boostProgram`
Enumeration completeness	`mem_allTypes_of_rank`, `mem_allTerms_of_rank`
Compiler rank and membership	`boostProgram_rank`, `boostProgram_mem`
Exact C6 baseline	`walk_original`, `rawIndex_initial`, `number_initial`
Closed and relative score bounds	`closed_program_lt_joinReflect`, `relative_program_lt_joinReflect`
Amplified join comparison	`boost_number_lt_join`
Bootstrap comparison	`boost_number_lt_bootstrap`
Exact tree-program retention	`tree_slot`
Amplifier properties	`boost_mono`, `self_lt_boost`, `self_le_boostIter`
Finite rounds	`boostIter_le_rounds`
Uniform tower margin	`iterated_boost_lt_final`
Closed entry comparison	`BusyLean.contender_6_lt_contender_7`
Closed 44-fold margin	`BusyLean.contender_7_tower_margin`
Eventual finite-round domination	`finite_round_program_lt_final`
Finite-round non-representability	`no_finite_round_program_for_final`
Limit-library non-representability	`no_limit_program_for_final`

B Provenance and source boundaries

The exact baseline commit is `86b360648163f0249eab79539438b6a83dfe8215`. The three vendored source files are independently identified by the following Git blob SHA-1 hashes:

Original language definitions `0cb1e14966d62d4c17302f8f6b41316fb4181a42`.
Original reflection definitions `2703943ffe1207bd398d5f750daadff8139c5418`.
Original Contender 6 definitions `248aa97049d0045b9635bb816842870d4a19eaed`.

The source audit recomputes each hash using Git's blob header and raw file bytes, not just a filename comparison. It additionally records SHA-256 hashes in `evidence/SOURCE_MANIFEST.json`. License

notices from BusyLean and the original Coq contest are retained. The repository was inspected but was not modified or pushed to during preparation of this package.

C A general amplification schema

The specific tower amplifier is not essential to the semantic mechanism. Let $A : \mathbb{N} \to \mathbb{N}$ be a computably defined monotone function with $x < A(x)$ for all x. Replace only the level-4 library value B by A, and use the same rank-17 syntax p_B (now better named p_A). The parametric compiler lemma gives $E_A(m, f, n, p_A) = A(\nu_m(n))$ without altering its rank. Repeating the finite-max, bootstrap, and round proofs gives

$$A^{\circ(n+2)}(C(n)) < N_A(n).$$

The constant A must itself be explicitly computable and must be included in any definition-size and compilation audit. It is not legitimate to insert an uncomputable "ideal amplifier" and retain the computability conclusion. This general statement is proved by the same mathematical argument; the packaged final number fixes $A = B$ and the Lean comparison modules are written for that fixed choice.

The schema explains why iterating a numerical transformer is a useful *certificate* rather than a complete description of the new number. Changing A changes a library value, but the surrounding reflection still ranges over arbitrary programs using the complete prior hierarchy and its generated trees.

References

[1] VladimirReshetnikov/BusyLean contributors. *BusyLean: name the biggest number, in Lean*; repository README and rules. Pinned revision `86b360648163f0249eab79539438b6a83dfe8215`. Repository README at the inspected revision.

[2] BusyLean contributors. *P5 finite-type W-language definitions.* `proposals/P5-w-reflection/lean/BigNumber/Definitions.lean`, same revision. Pinned language source.

[3] BusyLean contributors. *Reflection over the W-language* and accompanying inequalities. `BusyLean/Reflection.lean` and `BusyLean/ReflectionFacts.lean`, same revision. Pinned reflection source.

[4] BusyLean contributors. *Contender 6.* `BusyLean/Contender6.lean`, same revision. Pinned definition source.

[5] BusyLean contributors. *Contender 6: comparison theorems.* `BusyLean/Comparison.lean`, same revision. Pinned comparison source.

[6] BusyLean contributors. *Review of the five incoming proposals.* `proposals/REVIEW.md`, same revision. Pinned review. Its ordinal rankings are treated as informal estimates, not imported theorems.

[7] The Lean development team. *The Lean Language Reference: Recursive Definitions*, especially structural recursion and its translation to recursors. Consulted October 2026. https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/. The package itself pins Lean 4.34.0 rather than relying on the moving "latest" documentation version.